Learning from Data: A brief report on Classification, Retrieval and Feature Selection

Yanqiao Chen
Department of Computer Science and Engineering
Southern University of Science and Technology
12412115@mail.sustech.edu.cn

Abstract

In this report, we present a brief overview of three fundamental topics in machine learning: classification, retrieval, and feature selection. We compare various algorithms and techniques used in each area, including their advantages and disadvantages. We also discuss the importance of these topics in real-world applications and provide insights into future research directions.

I Introduction

High-dimensional feature representations are ubiquitous in modern computer vision and machine learning pipelines. Whether derived from hand-crafted descriptors or deep neural network embeddings, these vectors serve as the foundation for a wide range of downstream tasks, including classification, similarity search, and feature selection. In this report, we investigate three core problems in learning from high-dimensional image features: (1) **image classification**, where the goal is to assign a semantic label to a given feature vector; (2) **image retrieval**, where the objective is to identify the most similar instances in a repository; and (3) **feature selection**, where the aim is to identify a compact subset of informative dimensions while preserving predictive performance.

All three tasks operate on a shared 256-dimensional feature space extracted from image data. Task 1 is formulated as a 10-class supervised classification problem with approximately 50,000 training samples. Task 2 involves retrieving the 5 most similar images from a repository of 5,000 items using various distance metrics. Task 3 requires selecting no more than 30 features out of 256 such that a fixed pre-trained classifier maintains high accuracy. For each task, we first establish a baseline method and then systematically explore alternative algorithms and design choices to exceed baseline performance.

The remainder of this report is organized as follows. Section II describes the methodological foundations of the models and techniques evaluated across the three tasks. Section III presents the experimental setup, quantitative results, and detailed analysis. Section IV discusses broader insights, limitations, and connections between tasks. Finally, Section V summarizes our findings and suggests directions for future work.

II Methods

This section presents the detailed technical foundations of the methods evaluated in each task, including their mathematical formulations and algorithmic implementations.

II-A Task 1: Image Classification

II-A1 Baseline: Softmax Regression

The baseline is a hand-implemented multinomial logistic regression model. Given a feature vector $\mathbf{x} \in \mathbb{R}^d$ with $d = 256$, the model computes class probabilities via the softmax function:

$$P(y = c \mid \mathbf{x}; \mathbf{W}, \mathbf{b}) = \frac{\exp(z_c)}{\sum_{j=1}^{C} \exp(z_j)}, \quad \text{where} \quad z_c = \mathbf{w}_c^\top \mathbf{x} + b_c, \tag{1}$$

where $\mathbf{W} = [\mathbf{w}_1, \ldots, \mathbf{w}_C] \in \mathbb{R}^{d \times C}$ and $\mathbf{b} \in \mathbb{R}^C$ are the weight matrix and bias vector, respectively, and $C = 10$ is the number of classes.

The model is trained by minimizing the cross-entropy loss over N training samples:

$$\mathcal{L}(\mathbf{W}, \mathbf{b}) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} \mathbb{1}[y_i = c] \log P(y_i = c \mid \mathbf{x}_i). \tag{2}$$

The gradient of the loss with respect to the logits is:

$$\frac{\partial \mathcal{L}}{\partial z_c^{(i)}} = P(y_i = c \mid \mathbf{x}_i) - \mathbb{1}[y_i = c], \tag{3}$$

from which the parameter gradients follow by the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_c} = \frac{1}{N} \sum_{i=1}^{N} \left(P(y_i = c \mid \mathbf{x}_i) - \mathbb{1}[y_i = c]\right) \mathbf{x}_i, \quad \frac{\partial \mathcal{L}}{\partial b_c} = \frac{1}{N} \sum_{i=1}^{N} \left(P(y_i = c \mid \mathbf{x}_i) - \mathbb{1}[y_i = c]\right). \tag{4}$$

Parameters are updated via gradient descent with learning rate η:

$$\mathbf{w}_c \leftarrow \mathbf{w}_c - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}_c}, \quad b_c \leftarrow b_c - \eta \frac{\partial \mathcal{L}}{\partial b_c}. \tag{5}$$

The baseline uses $\eta = 0.1$ and 10,000 iterations. Prior to training, Z-score normalization is applied:

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}, \quad \text{where} \quad \mu_j = \frac{1}{N} \sum_{i=1}^{N} x_j^{(i)}, \quad \sigma_j = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (x_j^{(i)} - \mu_j)^2}, \tag{6}$$

with the convention $\sigma_j = 1$ if the variance is zero to avoid division by zero.

II-A2 Logistic Regression with L2 Regularization

The `sklearn` implementation solves an L2-regularized maximum likelihood problem. The objective function is:

$$\min_{\mathbf{W}, \mathbf{b}} \mathcal{L}(\mathbf{W}, \mathbf{b}) + \frac{\lambda}{2} \sum_{c=1}^{C} \|\mathbf{w}_c\|_2^2, \tag{7}$$

where $\lambda = 1/(2C)$ is the regularization strength. We experiment with $C \in \{0.5, 1.0, 2.0\}$, corresponding to $\lambda \in \{1.0, 0.5, 0.25\}$. The optimization is performed using the L-BFGS quasi-Newton method, which approximates the inverse Hessian to achieve faster convergence than vanilla gradient descent.

II-A3 Linear Support Vector Classification (LinearSVC)

LinearSVC solves the following primal optimization problem:

$$\min_{\mathbf{W}, \mathbf{b}} \frac{1}{2} \sum_{c=1}^{C} \|\mathbf{w}_c\|_2^2 + C \sum_{i=1}^{N} \sum_{c \neq y_i} \max\left(0, 1 - \mathbf{w}_{y_i}^\top \mathbf{x}_i - b_{y_i} + \mathbf{w}_c^\top \mathbf{x}_i + b_c\right), \tag{8}$$

where the first term is the L2 regularization and the second term is the multiclass hinge loss (Crammer-Singer formulation). The hyperparameter C controls the trade-off between margin maximization and classification error.

II-A4 Random Forest

Random Forest is an ensemble of T decision trees, where each tree t is trained on a bootstrap sample of the training data. At each split node, a random subset of m features is considered. The prediction for a test sample is obtained by majority voting:

$$\hat{y} = \arg\max_{c} \sum_{t=1}^{T} \mathbb{1}[h_t(\mathbf{x}) = c], \tag{9}$$

where $h_t(\mathbf{x})$ is the prediction of the t-th tree. We evaluate configurations with $T \in \{200, 500\}$ and maximum depth constraints $\in \{20, 30, \infty\}$ to control overfitting.

II-A5 Multi-Layer Perceptron (MLP)

The MLP is a feed-forward neural network with L hidden layers. For a single hidden layer with h units (MLP(h)), the forward pass is:

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}, \tag{10}$$

$$\mathbf{a}^{[1]} = \mathrm{ReLU}(\mathbf{z}^{[1]}) = \max(\mathbf{0}, \mathbf{z}^{[1]}), \tag{11}$$

$$\mathbf{z}^{[2]} = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}, \tag{12}$$

$$\hat{\mathbf{y}} = \mathrm{softmax}(\mathbf{z}^{[2]}), \tag{13}$$

where $\mathbf{W}^{[1]} \in \mathbb{R}^{h\times d}$, $\mathbf{b}^{[1]} \in \mathbb{R}^h$, $\mathbf{W}^{[2]} \in \mathbb{R}^{C\times h}$, and $\mathbf{b}^{[2]} \in \mathbb{R}^C$. For MLP(256, 128), an additional hidden layer is inserted with analogous computations.

The network is trained by backpropagation to minimize the cross-entropy loss. `sklearn`'s `MLPClassifier` uses the Adam optimizer by default. Early stopping is enabled: a validation set (10% of training data) monitors the loss, and training terminates when the validation score fails to improve for 10 consecutive epochs.

II-B Task 2: Image Retrieval

II-B1 Problem Formulation

Given a query set $\mathcal{Q} = \{\mathbf{q}_1, \ldots, \mathbf{q}_M\} \subset \mathbb{R}^d$ and a repository $\mathcal{R} = \{\mathbf{r}_1, \ldots, \mathbf{r}_N\} \subset \mathbb{R}^d$ with $N = 5{,}000$ and $d = 256$, the goal is to retrieve, for each query $\mathbf{q}_i$, the indices of the $k = 5$ most similar repository items:

$$\mathcal{N}_k(\mathbf{q}_i) = \arg\min_{\mathcal{I}\subset[N],|\mathcal{I}|=k} \sum_{j\in\mathcal{I}} d(\mathbf{q}_i, \mathbf{r}_j), \tag{14}$$

where $d(\cdot,\cdot)$ is a distance metric.

II-B2 Distance Metrics

The baseline uses Euclidean (L2) distance computed by a hand-written brute-force loop:

$$d_{\mathrm{L2}}(\mathbf{q}, \mathbf{r}) = \|\mathbf{q} - \mathbf{r}\|_2 = \sqrt{\sum_{j=1}^{d} (q_j - r_j)^2}. \tag{15}$$

We systematically evaluate four metrics. All are implemented via `sklearn.neighbors.NearestNeighbors` with `algorithm='brute'`, which computes the full $M \times N$ distance matrix and selects the k smallest entries per row.

Cosine Distance. The cosine similarity is the cosine of the angle between two vectors:

$$\mathrm{sim}_{\cos}(\mathbf{q}, \mathbf{r}) = \frac{\mathbf{q}^\top \mathbf{r}}{\|\mathbf{q}\|_2 \|\mathbf{r}\|_2} = \frac{\sum_{j=1}^{d} q_j r_j}{\sqrt{\sum_{j=1}^{d} q_j^2}\sqrt{\sum_{j=1}^{d} r_j^2}}. \tag{16}$$

The cosine distance used by `sklearn` is $d_{\cos}(\mathbf{q}, \mathbf{r}) = 1 - \mathrm{sim}_{\cos}(\mathbf{q}, \mathbf{r})$. Cosine distance is invariant to vector magnitude, making it suitable when the direction of the feature vector is more semantically meaningful than its scale.

Manhattan (L1) Distance:

$$d_{\mathrm{L1}}(\mathbf{q}, \mathbf{r}) = \|\mathbf{q} - \mathbf{r}\|_1 = \sum_{j=1}^{d} |q_j - r_j|. \tag{17}$$

L1 distance is more robust to outliers than L2 because it does not square the differences.

Correlation Distance. First, each vector is centered:

$$\bar{\mathbf{q}} = \mathbf{q} - \bar{q}\mathbf{1}, \quad \text{where} \quad \bar{q} = \frac{1}{d}\sum_{j=1}^{d} q_j. \tag{18}$$

The Pearson correlation coefficient is:

$$\rho(\mathbf{q}, \mathbf{r}) = \frac{\bar{\mathbf{q}}^\top \bar{\mathbf{r}}}{\|\bar{\mathbf{q}}\|_2 \|\bar{\mathbf{r}}\|_2}, \tag{19}$$

and the correlation distance is $d_{\text{corr}}(\mathbf{q}, \mathbf{r}) = 1 - \rho(\mathbf{q}, \mathbf{r})$.

Effect of Normalization. We also evaluate all metrics after Z-score normalizing both the repository and query features using repository statistics $(\boldsymbol{\mu}, \boldsymbol{\sigma})$. The normalized feature is $\tilde{\mathbf{x}} = (\mathbf{x} - \boldsymbol{\mu})/\boldsymbol{\sigma}$.

II-C Task 3: Feature Selection

II-C1 Problem Formulation

Let $\mathbf{X} \in \mathbb{R}^{N \times d}$ be the validation data with N samples and $d = 256$ features, and $\mathbf{y} \in \{0, \ldots, C-1\}^N$ be the labels. A fixed pre-trained Softmax classifier has parameters $(\mathbf{W}, \mathbf{b})$ where the augmented weight matrix $\mathbf{W}_{\text{aug}} \in \mathbb{R}^{(d+1) \times C}$ includes the bias as row 0. The goal is to find a binary mask $\mathbf{m} \in \{0, 1\}^d$ with $\|\mathbf{m}\|_0 \leq 30$ that maximizes validation accuracy:

$$\max_{\mathbf{m}} \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}\left[\arg\max_c \; f_c(\mathbf{x}_i \odot \mathbf{m}) = y_i\right], \quad \text{s.t.} \quad \|\mathbf{m}\|_0 \leq 30, \tag{20}$$

where $f_c(\tilde{\mathbf{x}}) = \mathbf{w}_c^\top \tilde{\mathbf{x}} + b_c$ is the logit for class c, and $\odot$ denotes element-wise multiplication.

II-C2 Wrapper Methods

LOO Backward Elimination. This greedy algorithm starts with all d features active and iteratively removes the least important feature until only 30 remain. The pseudocode is given in Algorithm 1.

Algorithm 1 LOO Backward Elimination with Precomputed Logits

```
Augment data: X_bias ← [1, X] ∈ R^{N×(d+1)}
Precompute full logits: ℓ ← X_bias W_aug ∈ R^{N×C}
A ← {0, 1, ..., d − 1}, m ← 1_d
while |A| > 30 do
    j* ← arg max_{j∈A} Acc(ℓ − X_bias[:, j + 1] · W_aug[j + 1, :])
    ℓ ← ℓ − X_bias[:, j* + 1] · W_aug[j* + 1, :]          ▷ Permanent removal
    m_{j*} ← 0, A ← A \ {j*}
end while
return m
```

The key acceleration technique exploits the linearity of the Softmax model. The logit for class c without feature j is:

$$\ell_c^{(-j)}(\mathbf{x}_i) = b_c + \sum_{p \neq j} w_{p,c} \cdot x_{i,p} = \ell_c(\mathbf{x}_i) - w_{j,c} \cdot x_{i,j}. \tag{21}$$

In matrix form, removing feature j corresponds to subtracting the outer product:

$$\boldsymbol{\ell}^{(-j)} = \boldsymbol{\ell} - \mathbf{X}_{\text{bias}}[:, j+1] \otimes \mathbf{W}_{\text{aug}}[j+1, :], \tag{22}$$

where $\otimes$ denotes the outer product. This reduces the per-evaluation cost from $O(NdC)$ to $O(NC)$, avoiding repeated matrix multiplications.

The accuracy function is:

$$\mathrm{Acc}(\boldsymbol{\ell}) = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}\left[\arg\max_{c} \ell_{i,c} = y_i\right]. \tag{23}$$

Forward Selection. This complementary greedy approach starts with an empty feature set and iteratively adds the most beneficial feature:

Algorithm 2 Forward Selection

```
S ← ∅, R ← {0, 1, ..., d − 1}, m ← 0_d
while |S| < 30 do
    j* ← arg max_{j∈R} Acc(X_bias[:, {0} ∪ S ∪ {j}] · W_aug[{0} ∪ S ∪ {j}, :])
    S ← S ∪ {j*}, R ← R \ {j*}, m_{j*} ← 1
end while
return m
```

Unlike backward elimination, forward selection requires a full matrix multiplication in each iteration because the active feature set grows. However, it only runs for 30 iterations, making it faster overall.

Approximate Shapley Value. The Shapley value of feature j measures its average marginal contribution across all possible feature subsets. Formally, for a valuation function $v : 2^{[d]} \to \mathbb{R}$ (here, $v(S) = \mathrm{Acc}(\mathbf{m}_S)$ where $\mathbf{m}_S$ selects exactly S), the Shapley value is:

$$\phi_j = \sum_{S \subseteq [d]\setminus\{j\}} \frac{|S|!(d-|S|-1)!}{d!}\left[v(S \cup \{j\}) - v(S)\right]. \tag{24}$$

Exact computation requires evaluating all 2^d subsets, which is infeasible for $d = 256$. We approximate ϕ_j via Monte Carlo sampling of random permutations. For each permutation π, we traverse features in order and record the accuracy when feature j is added:

$$\hat{\phi}_j = \frac{1}{K}\sum_{k=1}^{K}\left[v(S_{\pi_k,j} \cup \{j\}) - v(S_{\pi_k,j})\right], \tag{25}$$

where $S_{\pi_k,j}$ is the set of features preceding j in permutation π_k, and $K = 500$ is the number of sampled permutations. Features with the top 30 $\hat{\phi}_j$ values are selected.

II-C3 Filter Methods

Filter methods select features based on statistical criteria independent of the fixed classifier's predictions.

Mutual Information. The mutual information between feature j and the class label Y is:

$$\mathrm{MI}(X_j, Y) = \sum_{x \in \mathcal{X}_j}\sum_{y \in \mathcal{Y}} P(x,y)\log\frac{P(x,y)}{P(x)P(y)}, \tag{26}$$

where $\mathcal{X}_j$ and $\mathcal{Y}$ are the domains of feature j and the label, respectively. Since the features are continuous, we use the k-nearest neighbor estimator implemented in `sklearn.feature_selection.mutual_info_class` with $k = 3$. Features are ranked by $\mathrm{MI}(X_j, Y)$ and the top 30 are retained.

Pearson Correlation. For each feature j, we compute the sample correlation coefficient with the integer-encoded label:

$$r_j = \frac{\sum_{i=1}^{N}(x_{i,j} - \bar{x}_j)(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{N}(x_{i,j} - \bar{x}_j)^2}\sqrt{\sum_{i=1}^{N}(y_i - \bar{y})^2}}, \tag{27}$$

and select features with the largest $|r_j|$.

Weight-Based Methods. These heuristics assume that dimensions with larger weights in the pre-trained model are more important. For feature j (corresponding to weight row $j + 1$ in $\mathbf{W}_{\mathrm{aug}}$):

- **Weight Magnitude**: $\text{score}_j = \sum_{c=1}^{C} |W_{j+1,c}|$.
- **Weight L2 Norm**: $\text{score}_j = \sqrt{\sum_{c=1}^{C} W_{j+1,c}^2}$.

Features with the top 30 scores are selected.

III Experiment

III-A Experimental Setup

All experiments were conducted using Python 3.10 with NumPy 1.26.1 and scikit-learn 1.5.2. Each task follows a consistent preprocessing pipeline: the first column of every data file (an index/ID) is discarded, leaving 256-dimensional feature vectors.

Task 1 – Classification: The training set contains 49,976 samples. We apply Z-score normalization using training-set statistics (mean and standard deviation per dimension), with zero-variance dimensions set to a safe divisor of 1.0. An 80/20 train-validation split is performed with random seed 123, matching the baseline notebook. The final model is retrained on the full normalized dataset before submission.

Task 2 – Retrieval: The repository contains 5,000 items with 256 features each. Since no labeled validation set is provided for retrieval, we use self-retrieval on the first 1,000 repository samples as a proxy to compare metrics. Performance is measured by top-5 overlap with the L2 baseline, inference time, and self-hit rate.

Task 3 – Feature Selection: A fixed validation set with known labels and a pre-trained weight matrix (257 x 10, including bias) are provided. The constraint is strictly enforced: no more than 30 features may be selected. Validation accuracy on the provided data is used as the optimization objective.

III-B Results

III-B1 Task 1: Image Classification

Table I summarizes the validation accuracy of all evaluated classifiers.

TABLE I: Validation Accuracy on Task 1 (Image Classification)

Model	Validation Accuracy
Baseline (Softmax Regression, hand-coded)	51.15%
LogisticRegression (L-BFGS, $C = 0.5$)	51.32%
LogisticRegression (L-BFGS, $C = 1.0$)	51.31%
LogisticRegression (L-BFGS, $C = 2.0$)	51.31%
LogisticRegression (SAGA, $C = 1.0$)	51.29%
LinearSVC ($C = 1.0$)	50.66%
RandomForest ($n = 200$, max_depth=20)	48.00%
RandomForest ($n = 500$, max_depth=30)	49.97%
RandomForest ($n = 200$, unlimited depth)	46.62%
MLP (128 hidden units)	**53.60%**
MLP (256, 128 hidden units)	53.82%

III-B2 Task 2: Image Retrieval

Table II compares distance metrics and preprocessing strategies. All methods achieve 100% top-5 self-hit because every query is itself a member of the repository. The key discriminative metric is the overlap rate with the L2 (raw) baseline, which indicates how similar the ranking is to the reference implementation.

III-B3 Task 3: Feature Selection

Table III reports the validation accuracy of all seven feature selection methods. The baseline (random selection of 30 features) achieves only 16.49%.

TABLE II: Retrieval Performance Comparison on 1,000 Self-Queries

Method	Inference Time (s)	Overlap with L2 (raw)
L2 (raw) – baseline-like	1.6711	—
Cosine (raw)	**0.0815**	**99.68%**
Manhattan L1 (raw)	0.1057	72.78%
Correlation (raw)	0.6376	93.84%
L2 (normalized)	0.0155	38.96%
Cosine (normalized)	0.0802	43.40%
Manhattan L1 (normalized)	0.1000	39.96%
Correlation (normalized)	0.6231	43.04%

TABLE III: Validation Accuracy on Task 3 (Feature Selection, max 30 features)

Method	Validation Accuracy	Time (s)
Baseline (Random)	16.49%	—
Weight Magnitude	28.27%	<0.01
Weight L2 Norm	30.47%	<0.01
Pearson Correlation	31.07%	0.02
Mutual Information	39.98%	5.73
Forward Selection	45.56%	9.38
Approximate Shapley Value (500 perm)	45.77%	89.53
LOO Backward Elimination	**45.94%**	14.49

III-C Analysis

Task 1 – Classification: The results reveal a clear hierarchy. Linear models (Softmax baseline, LogisticRegression, LinearSVC) cluster around 50–51.3%, suggesting that a linear decision boundary is insufficient for this dataset. Random Forest performs worse (46–50%), indicating that tree-based ensembles struggle with the high-dimensional, dense real-valued features; the unlimited-depth variant even overfits severely (100% training accuracy vs 46.62% validation). The MLP models break through this ceiling, with MLP(128) achieving 53.60% and MLP(256,128) reaching 53.82%. The marginal gain from adding a second hidden layer (0.22%) does not justify the increased complexity and overfitting risk, so MLP(128) is selected as the final model.

Task 2 – Retrieval: Cosine distance on raw features achieves a 99.68% overlap with the L2 baseline while being approximately 20× faster (0.08 s vs 1.67 s). This near-perfect alignment implies that, for this feature space, vector orientation is the dominant factor in similarity, and the L2 ranking is effectively approximated by cosine ranking. Feature normalization dramatically alters the ranking (overlap drops to 38–43%), but without labeled ground truth, it is unclear whether this represents an improvement or degradation. Given that the grading criterion requires exceeding the baseline accuracy, Cosine (raw) is the safest choice because it nearly replicates the baseline behavior with far superior computational efficiency.

Task 3 – Feature Selection: The results demonstrate a stark divide between wrapper and filter methods. The top three methods (LOO Backward, Shapley, Forward Selection) all exceed 45.5%, whereas the best filter method (Mutual Information) reaches only 39.98%. This gap highlights a critical principle: when the evaluation metric is model accuracy, methods that directly optimize that metric (wrappers) outperform heuristic statistical criteria (filters). The fixed model weights, which were trained on all 256 dimensions, do not reliably indicate which subset of 30 dimensions is optimal – explaining the poor performance of Weight Magnitude and Weight L2 Norm. LOO Backward Elimination slightly outperforms Forward Selection (45.94% vs 45.56%) despite both being greedy wrapper methods; this may be because starting from the full

set preserves synergistic feature interactions that Forward Selection discovers too late.

IV Discussion

General Patterns. Across all three tasks, a consistent theme emerges: *model-aware, task-specific optimization outperforms generic heuristics*. In Task 1, introducing non-linearity via hidden layers (MLP) directly addresses the limitation of linear models. In Task 2, choosing a metric that aligns with the implicit structure of the feature space (cosine) yields both accuracy and speed. In Task 3, wrapper methods that use the actual classifier's feedback dramatically outperform filter methods that ignore the model.

Connections Between Tasks. Although the three tasks are formulated independently, they operate on the same underlying 256-dimensional image feature space, offering complementary perspectives on learning from high-dimensional data. Task 1 asks "what is this?"; Task 2 asks "what looks like this?"; and Task 3 asks "which dimensions matter?". The success of MLP in Task 1 suggests that the feature space contains non-linearly separable structure; the effectiveness of cosine similarity in Task 2 suggests that directions (angles) in this space carry more semantic information than raw distances; and the wrapper-based selection in Task 3 confirms that only a small fraction of dimensions (approximately 12%) are needed to preserve much of the discriminative information.

Limitations. Our experiments have several limitations. First, hyperparameter tuning was conducted manually rather than via systematic grid search or Bayesian optimization, due to time constraints. Second, the retrieval evaluation relied on self-retrieval without ground-truth labels, so the overlap-based proxy may not perfectly correlate with the hidden test accuracy. Third, the feature selection task used a fixed linear classifier; the optimal 30-feature subset for a non-linear model (e.g., the MLP from Task 1) might differ substantially.

Practical Lessons. (1) Strong library implementations (e.g., `sklearn`) often provide order-of-magnitude speedups over naïve implementations without sacrificing accuracy. (2) When the evaluation metric is known and fixed, optimizing it directly (wrapper methods) is almost always better than relying on proxy heuristics (filter methods). (3) Simplicity and marginal gains must be carefully traded off: MLP(128) was preferred over MLP(256,128) because the 0.22% accuracy improvement did not justify the added complexity.

V Conclusion

In this report, we presented a systematic empirical study of classification, retrieval, and feature selection on a shared 256-dimensional image feature dataset. For each task, we established a baseline, explored multiple algorithmic alternatives, and selected the best-performing approach.

For **image classification**, an MLP with one hidden layer of 128 neurons achieved 53.60% validation accuracy, outperforming linear models by over 2 percentage points and demonstrating the value of non-linear representations. For **image retrieval**, cosine similarity on raw features replicated the L2 baseline ranking with 99.68% overlap while accelerating inference by 20×, making it both accurate and efficient. For **feature selection**, Leave-One-Out backward elimination raised validation accuracy from a random baseline of 16.49% to 45.94%, confirming that wrapper methods which leverage model feedback are superior to statistical filter heuristics when the goal is to maximize accuracy under a strict cardinality constraint.

These results underscore fundamental principles in applied machine learning: match model complexity to data complexity, align similarity metrics with feature-space geometry, and optimize selection criteria directly for the target metric. Future work could extend these findings by exploring deep metric learning for retrieval, conducting exhaustive hyperparameter searches, and investigating whether the selected 30-feature subset remains effective for non-linear classifiers.